

Bối cảnh

Các concert âm nhạc lớn tại Việt Nam — như **Anh Trai Say Hi, Anh Trai Vượt Ngàn Chông Gai, Em Xinh Say Hi, Chị Đẹp Đạp Gió Rẽ Sóng** — thu hút hàng chục nghìn khán giả. Khi ban tổ chức mở bán vé, website thường sập trong vài phút đầu do lượng truy cập đồng thời quá lớn; khán giả bị trừ tiền nhưng không nhận được vé; scalper dùng bot mua hết vé trong vài giây rồi bán lại với giá gấp nhiều lần.

Hiện tại nhiều sự kiện vẫn bán vé qua các kênh rời rạc: Zalo OA, Google Form, chuyển khoản thủ công — không đảm bảo tính công bằng và rất dễ xảy ra gian lận.

Công ty tổ chức sự kiện muốn xây dựng hệ thống **TicketBox** để số hóa toàn bộ quy trình bán vé, từ lúc mở bán đến khi khán giả vào cổng sự kiện.

Người dùng

Nhóm	Mô tả
Khán giả	Xem thông tin concert, mua vé, nhận e-ticket, check-in tại cổng
Ban tổ chức	Tạo và quản lý concert, cấu hình loại vé, theo dõi doanh thu và lượng bán
Nhân sự soát vé	Xác nhận vé tại cổng vào bằng mobile app

Yêu cầu hệ thống

Xem và mua vé

Khán giả có thể **xem danh sách các concert** sắp diễn ra, bao gồm **thông tin nghệ sĩ biểu diễn, địa điểm tổ chức, sơ đồ chỗ ngồi** (sơ đồ SVG tương tác theo khu: GA, SVIP, VIP, CAT1, CAT2) và **số vé còn lại theo thời gian thực** cho từng loại.

Khán giả chọn loại vé và số lượng, sau đó tiến hành **thanh toán qua cổng thanh toán** (VNPAY, MoMo). Sau khi thanh toán thành công, khán giả **nhận e-ticket dưới dạng mã QR** dùng để vào cổng sự kiện.

Mỗi tài khoản chỉ được mua tối đa một số lượng vé nhất định cho mỗi loại vé, do ban tổ chức cấu hình khi tạo concert (ví dụ: SVIP tối đa 2 vé/tài khoản, CAT1 tối đa 4 vé/tài khoản). Giới hạn này **áp dụng trên toàn bộ các đơn hàng đã thanh toán thành công** — khán giả không thể lách bằng cách tạo nhiều đơn hàng nhỏ.

Thông báo

Sau khi mua vé thành công, khán giả **nhận thông báo xác nhận qua app và email kèm e-ticket**. Khi concert sắp diễn ra (trước 24 giờ), hệ thống gửi **nhắc nhở tự động**. Hệ thống **cần được thiết kế để dễ dàng bổ sung kênh thông báo mới** (ví dụ: Zalo OA, SMS) trong tương lai mà không cần thay đổi lớn.

Quản trị

Ban tổ chức dùng trang web admin để **tạo concert mới, cấu hình các loại vé (tên, giá, số lượng, thời điểm mở bán), cập nhật thông tin, hoặc hủy concert**. Trang admin chỉ dành cho nội bộ và cần kiểm soát truy cập chặt chẽ — ba nhóm người dùng có quyền hạn khác nhau: khán giả chỉ có thể xem thông tin và mua vé; ban tổ chức có quyền tạo, sửa, hủy concert và **xem thống kê doanh thu**; nhân sự soát vé chỉ có quyền truy cập chức năng quét mã QR.

Soát vé tại sự kiện

Nhân sự tại cổng vào dùng **mobile app để quét mã QR trên e-ticket của khán giả**. Các địa điểm tổ chức concert lớn (sân vận động, nhà thi đấu) thường có vùng sáng không ổn định khi hàng chục nghìn người tập trung — app **phải cho phép ghi nhận soát vé tạm thời khi không có mạng và tự đồng bộ lại khi kết nối được phục hồi**.

AI Artist Bio

Ban tổ chức có thể tải lên file PDF hồ sơ nghệ sĩ hoặc press kit của concert. Hệ thống **tự động xử lý, tách nội dung, làm sạch văn bản và gửi sang mô hình AI để tạo bản giới thiệu ngắn gọn hiển thị trên trang chi tiết concert**.

Đồng bộ danh sách khách mời VIP

Một số concert có khu vực Guest List dành cho khách mời của nhãn hàng tài trợ. Hệ thống quản lý khách mời của nhãn hàng không có API — cách duy nhất là **nhận file CSV mà nhãn hàng gửi vào ban đêm trước ngày diễn**. TicketBox cần định kỳ nhập danh sách này để nhân sự soát vé có thể xác nhận khách mời tại cổng VIP.

Các vấn đề cần giải quyết

Tranh chấp vé: Một số loại vé SVIP của concert Anh Trai Say Hi chỉ có 200 chỗ nhưng có thể có hàng chục nghìn khán giả cố mua cùng lúc ngay khi mở bán. **Hệ thống phải đảm bảo không có hai khán giả nào cùng nhận được vé cuối cùng.**

Tải trọng đột biến: Khi concert Chị Đẹp Đạp Gió Rẽ Sóng mở bán, dự kiến khoảng 80.000 người truy cập trong 5 phút đầu, trong đó 70% dồn vào phút đầu tiên. Hệ thống **cần có cơ chế bảo vệ backend API khỏi bị quá tải, ngăn chặn bot và client gửi request liên tục, đồng thời đảm bảo tính công bằng giữa các khán giả thật.**

Thanh toán không ổn định: Nếu cổng thanh toán (VNPay/MoMo) gặp sự cố, khán giả vẫn phải xem được thông tin concert và danh sách vé còn lại bình thường. **Luồng mua vé có phí cần xử lý tình huống thanh toán timeout mà không gây ra trừ tiền hai lần; các tính năng không liên quan đến thanh toán vẫn phải hoạt động bình thường khi cổng thanh toán gặp sự cố kéo dài.**

Soát vé offline: Nhân sự ở khu vực sóng yếu trong sân vận động vẫn phải soát vé được cho khán giả; dữ liệu không được mất khi kết nối trở lại và không được cho phép một vé vào cổng hai lần.

Tích hợp một chiều: Không thể gọi API hệ thống quản lý khách mời của nhãn hàng — chỉ có thể đọc CSV được gửi theo lịch cố định. **Luồng nhập dữ liệu phải xử lý được file lỗi, dữ liệu trùng và không làm gián đoạn hệ thống đang chạy.**

Giới hạn vé per-user khó enforce dưới tải cao: Khi hàng chục nghìn người mua vé cùng lúc, **cần đảm bảo giới hạn số vé mỗi tài khoản được áp dụng chính xác — không để một người mua vượt quá giới hạn dù gửi nhiều request đồng thời.** Đây là bài toán tương tự tranh chấp chỗ ngồi nhưng ở phạm vi per-user thay vì toàn hệ thống.

Trang chủ và trang chi tiết concert bị quá tải: Trang danh sách concert và trang chi tiết từng concert bị đọc với tần suất rất cao (hàng nghìn lần/giây

trong giờ cao điểm) nhưng dữ liệu thay đổi không thường xuyên. Nếu mỗi request đều truy vấn thẳng vào database, hệ thống sẽ không chịu được tải. Cần có **chiến lược cache hợp lý để giảm tải cho database mà vẫn đảm bảo dữ liệu đủ cập nhật** (ví dụ: số vé còn lại phải phản ánh gần đúng thực tế).

Các nội dung cần thực hiện

Phần 1 — Blueprint

1. Tài liệu thiết kế hệ thống

Mô tả kiến trúc tổng thể của hệ thống, bao gồm các thành phần chính, cách chúng giao tiếp và lý do lựa chọn kiến trúc đó. Tài liệu cần trả lời được các câu hỏi: hệ thống gồm những phần nào, phần nào nói chuyện với nhau như thế nào, và khi một phần gặp sự cố thì phần còn lại bị ảnh hưởng ra sao.

2. C4 Diagram

Vẽ hai cấp độ đầu của C4 diagram:

- **Level 1 – System Context:** thể hiện TicketBox trong bức tranh toàn cảnh — ai dùng hệ thống, hệ thống ngoài nào được tích hợp.
- **Level 2 – Container:** phân rã hệ thống thành các container (ví dụ: web app, mobile app, backend API, database, message broker), chỉ rõ công nghệ đề xuất và cách các container giao tiếp với nhau.

3. High-Level Architecture Diagram

Vẽ sơ đồ kiến trúc tổng quan thể hiện luồng dữ liệu và sự phụ thuộc giữa các thành phần, đặc biệt ở các điểm tích hợp (cổng thanh toán, AI model, hệ thống khách mời CSV) và luồng soát vé offline.

4. Thiết kế cơ sở dữ liệu

Xác định các loại dữ liệu chính trong hệ thống, đề xuất loại database phù hợp (SQL, NoSQL, hoặc kết hợp) và giải thích lý do lựa chọn dựa trên đặc điểm của từng loại dữ liệu. Thiết kế schema cho các entity quan trọng nhất.

5. Mô tả các luồng nghiệp vụ quan trọng

Mô tả chi tiết ít nhất hai trong số các luồng sau:

- Luồng mua vé (từ khi bấm “Mua vé” đến khi nhận e-ticket)
- Luồng soát vé khi mất mạng và đồng bộ lại
- Luồng nhập danh sách khách mời từ CSV

Với mỗi luồng, trình bày các bước xử lý, các thành phần tham gia và cách hệ thống phản ứng khi có lỗi xảy ra giữa chừng.

6. Thiết kế kiểm soát truy cập

Thiết kế mô hình phân quyền cho hệ thống. Xác định các nhóm người dùng, quyền hạn tương ứng với từng nhóm, và giải thích cách hệ thống kiểm tra quyền tại từng điểm truy cập (API endpoint, trang admin, mobile app soát vé). Nhóm có thể tham khảo mô hình **RBAC (Role-Based Access Control)** hoặc đề xuất cách tiếp cận khác nếu có lý do phù hợp.

7. Thiết kế các cơ chế bảo vệ hệ thống

Với mỗi vấn đề kỹ thuật dưới đây, trình bày giải pháp nhóm lựa chọn, giải thích cách nó hoạt động và lý do phù hợp với bài toán. Các kỹ thuật được gợi ý nhưng nhóm có thể đề xuất giải pháp thay thế nếu lập luận thuyết phục:

- **Kiểm soát tải đột biến:** Làm thế nào để backend API không bị quá tải khi 80.000 người cùng truy cập mua vé trong phút đầu mở bán? (gợi ý: *Rate Limiting — Fixed Window, Sliding Window, Token Bucket, Leaky Bucket*)
- **Xử lý cổng thanh toán không ổn định:** Làm thế nào để hệ thống phản ứng khi VNPay/MoMo liên tục lỗi mà không kéo sập toàn bộ dịch vụ? (gợi ý: *Circuit Breaker với các trạng thái Closed / Open / Half-Open, kết hợp Graceful Degradation*)
- **Chống trùng tiền hai lần:** Làm thế nào để đảm bảo một giao dịch mua vé chỉ được thực hiện đúng một lần dù khán giả bấm mua nhiều lần hoặc mạng bị ngắt giữa chừng? (gợi ý: *Idempotency Key — cơ chế sinh key, nơi lưu trữ, cách kiểm tra trùng lặp, thời gian hết hạn*)
- **Caching:** Làm thế nào để trang danh sách concert và trang chi tiết không làm quá tải database khi có hàng nghìn request/giây, trong khi vẫn phản ánh đúng số vé còn lại? (gợi ý: *Cache-aside với Redis — xác định TTL phù hợp cho từng loại dữ liệu: thông tin concert ít thay đổi có thể cache lâu, số vé còn lại cần TTL ngắn hoặc invalidate chủ động khi có giao dịch thành công*)

Phần 2 — Cài đặt

Phần mềm hoàn chỉnh, có thể chạy được, cài đặt toàn bộ hệ thống đã mô tả trong Blueprint. Phần cài đặt phải bao gồm:

- **Tính năng nghiệp vụ đầy đủ:** Tất cả các chức năng được mô tả trong phần Yêu cầu hệ thống — xem concert, mua vé, thông báo, quản trị, soát vé, AI Artist Bio, đồng bộ CSV khách mời.
- **Các cơ chế kỹ thuật:** Toàn bộ giải pháp đã thiết kế trong Blueprint mục 6 và 7 phải được cài đặt thực sự trong code, không chỉ mô phỏng hoặc stub.
- **Hướng dẫn khởi chạy:** README rõ ràng, đủ để người chấm có thể clone repository và chạy được hệ thống mà không cần hỏi thêm.
- **Dữ liệu mẫu:** Seed data hoặc script tạo dữ liệu ban đầu — bao gồm các concert mẫu (Anh Trai Say Hi, Anh Trai Vượt Ngàn Chông Gai, Em Xinh Say Hi, Chị Đẹp Đạp Gió Rẽ Sóng) với đầy đủ loại vé, giá và sơ đồ chỗ ngồi — để có thể thao tác và kiểm tra ngay sau khi khởi chạy.

Tham khảo: Template Blueprint

Template tham khảo theo cấu trúc của [OpenSpec](#) — framework spec-driven development, gồm ba lớp tài liệu: **proposal** (vấn đề và lý do), **design** (giải pháp kỹ thuật), **specs** (kịch bản và ràng buộc cho từng tính năng). Nhóm có thể bổ sung mục hoặc điều chỉnh cấu trúc nếu phù hợp.

```
blueprint/  
├─ proposal.md          # Bối cảnh, vấn đề, mục tiêu  
├─ design.md            # Kiến trúc, sơ đồ, quyết định  
                           kỹ thuật  
└─ specs/  
    ├─ auth.md          # Đặc tả phân quyền  
    ├─ payment.md        # Đặc tả luồng thanh toán và  
                           chống trừ tiền 2 lần  
    ├─ checkin.md        # Đặc tả luồng soát vé offline  
    └─ ...                # Đặc tả các tính năng khác
```

proposal.md

```
# TicketBox — Project Proposal  
  
## Vấn đề  
  
<!-- Mô tả vấn đề hiện tại mà hệ thống cần giải quyết.  
     Tại sao các kênh bán vé hiện tại (Zalo OA, Google  
     Form, chuyển khoản) không còn đủ?
```

Hậu quả cụ thể: website sập, trừ tiền không ra vé, scalper bot vét hết vé. -->

Mục tiêu

<!-- Hệ thống cần đạt được gì? Định lượng nếu có thể.

Ví dụ: hỗ trợ 80.000 người truy cập trong 5 phút đầu mở bán mà không sập. -->

Người dùng và nhu cầu

<!-- Ai dùng hệ thống? Họ cần làm gì? Điều gì quan trọng nhất với họ? -->

Phạm vi

<!-- Những gì thuộc phạm vi đề án này.

Những gì KHÔNG thuộc phạm vi (ví dụ: tích hợp payment gateway thật, hạ tầng production). -->

Rủi ro và ràng buộc

<!-- Các vấn đề kỹ thuật đã biết trước: tranh chấp vé, tải đột biến,

cổng thanh toán không ổn định, soát vé offline, tích hợp một chiều CSV. -->

design.md

TicketBox – Technical Design

Kiến trúc tổng thể

<!-- Mô tả architectural style được chọn và lý do.

Hệ thống gồm những thành phần nào? Chúng giao tiếp với nhau như thế nào? -->

C4 Diagram

Level 1 – System Context

<!-- Sơ đồ: TicketBox + actors + hệ thống ngoài (VNPAY, MoMo, AI model, CSV nhân hàng) -->

Level 2 – Container

<!-- Sơ đồ: web app, mobile app soát vé, backend API, database, message broker, ... -->

```
## High-Level Architecture Diagram
<!-- Sơ đồ luồng dữ liệu, đặc biệt tại các điểm tích hợp
và luồng soát vé offline -->

## Thiết kế cơ sở dữ liệu
<!-- Loại database, lý do lựa chọn, schema các entity
chính -->

## Thiết kế kiểm soát truy cập
<!-- Mô hình phân quyền, các nhóm người dùng, cách kiểm
tra quyền tại từng điểm truy cập -->

## Thiết kế các cơ chế bảo vệ hệ thống

### Kiểm soát tải đột biến
<!-- Giải pháp, thuật toán, ngưỡng, hành vi khi vượt
ngưỡng -->

### Xử lý cổng thanh toán không ổn định
<!-- Giải pháp, các trạng thái, ngưỡng kích hoạt, hành vi
khi lỗi -->

### Chống trừ tiền hai lần
<!-- Cơ chế, nơi lưu trữ, TTL, luồng xử lý khi phát hiện
trùng lặp -->

### Caching
<!-- Xác định các đối tượng cần cache (danh sách concert,
chi tiết concert, số vé còn lại).
Chiến lược: Cache-aside, Write-through hay
Write-back?
TTL cho từng loại. Cách invalidate khi dữ liệu thay
đổi (đặc biệt: số vé sau mỗi giao dịch). -->

## Các quyết định kỹ thuật quan trọng (ADR)
<!-- Với mỗi quyết định lớn: lựa chọn gì, tại sao, đánh
đổi gì.
Ví dụ: SQL vs NoSQL, JWT vs Session, Kafka vs
RabbitMQ, optimistic vs pessimistic locking, ... -->
```

[specs/\[feature\].md](#)


```
# Đặc tả: [Tên tính năng]
```

```
## Mô tả
```

```
<!-- Tính năng này làm gì? -->
```

```
## Luồng chính
```

```
<!-- Các bước xử lý theo thứ tự, các thành phần tham gia -->
```

```
## Kịch bản lỗi
```

```
<!-- Điều gì xảy ra khi: timeout, mất mạng, dữ liệu không hợp lệ, ... -->
```

```
## Ràng buộc
```

```
<!-- Giới hạn hiệu năng, bảo mật, tính nhất quán cần đảm bảo -->
```

```
## Tiêu chí chấp nhận
```

```
<!-- Làm thế nào để biết tính năng này hoạt động đúng? -->
```

Quy định nộp bài

Định dạng file nộp

- Mỗi nhóm nộp một file text duy nhất lên hệ thống.
- Tên file: mã-nhóm_mssv1_mssv2_mssv3_mssv4.txt (ví dụ: N01_21127001_21127002_21127003_21127004.txt).
- Nội dung file: **Link Google Drive** public chứa tất cả các thành phần bài làm.

Cấu trúc thư mục trên Google Drive

Thư mục Drive của nhóm phải bao gồm đủ ba thành phần:

1. **Blueprint** — Nhóm có thể nộp theo một trong hai hình thức:
 - **PDF:** Một file `blueprint.pdf` duy nhất chứa đầy đủ các thành phần theo template.
 - **Markdown:** Thư mục `blueprint/` tổ chức theo cấu trúc template, upload trực tiếp lên Drive.

2. **Source code** — Thư mục `src/` chứa toàn bộ mã nguồn, kèm thư mục `data/` chứa seed data và script khởi tạo cơ sở dữ liệu, và file `README.md` với hướng dẫn cài đặt và khởi chạy.
3. **Video trình bày** — Thư mục `clips/` chứa video quay màn hình trình bày các vấn đề kỹ thuật mà nhóm đã giải quyết (không cần slide). Nội dung phải bao gồm **camera thành viên thuyết trình** và **demo trực tiếp trên code hoặc ứng dụng đang chạy**. Quy định kỹ thuật: độ phân giải **FullHD (1080p)**, bitrate khoảng **720 kbps**, định dạng MP4.